



MediGuide

SCRIPT DE SOUTENANCE · 15 MINUTES · SAE BUT3 S6

Script oral

Chatbot médical RAG

Le texte de chaque intervenant, slide par slide, avec timing indicatif et passages de parole. À lire en mode présentateur PowerPoint.

ÉQUIPE

Yousra · Imane · Yannis · Grace

ENCADREMENT

Mr FAYE & Mme AZZAG

DÉMO LIVE

sae.amanawebagency.com

DURÉE TOTALE

~15 minutes (+ Q&R)

Planning oral

Qui	Partie	Slides	Durée
TOUS	Intro	1	~30 s
TOUS	Sommaire	2	~30 s
GRACE	Contexte & Architecture	3 – 7	~3 min
YOUSRA	Traitement des PDFs & Pipeline RAG	8 – 13	~3 min
IMANE	Comparaison encodeurs & DuckDuckGo	14 – 18	~3 min
YANNIS	Hermes, Déploiement & Démo	19 – 26	~4 min
TOUS	Conclusion & Questions	27 – 28	~1 min

Code couleur utilisé tout au long du script — chaque intervenant a sa couleur. À l'oral, regardez votre couleur pour savoir si c'est votre tour.

■ Intro & Sommaire

Tous les 4 ensemble · slides 1 et 2

01

TOUS · Intro

~30 s

Slide 1 — « Chatbot médical RAG » (page de titre)

Bonjour à tous. Nous c'est **Yousra, Imane, Yannis et Grace**.

On va vous présenter notre projet de SAE : un chatbot médical basé sur une approche RAG.

On a 15 minutes pour vous présenter ça, et après on répondra à vos questions.

Tout est déployé en prod, vous pouvez le tester sur **sae.amanawebagency.com** ou flasher le QR code à la fin.

💡 *Présentation simultanée par les 4 si possible (chacun se présente brièvement).*

02

TOUS · Sommaire

~30 s

Slide 2 — Sommaire en 5 sections

Voilà ce qu'on va couvrir aujourd'hui en 4 grandes parties.

Grace commence avec le contexte et l'architecture.

Yousra enchaîne avec le traitement des PDFs et le pipeline RAG.

Imane prend ensuite la comparaison d'encodeurs et le bonus DuckDuckGo.

Yannis finit avec Hermes, le déploiement et la démo live.

On revient tous à la fin pour les questions.

PASSE LA PAROLE À : GRACE

■ Partie 1 — Contexte & Architecture

Grace · slides 3 à 7 · environ 3 minutes

03 GRACE

~10 s

Slide 3 — Transition « Partie 1 — Contexte & Architecture »

Slide de transition. Je laisse **5 secondes au jury pour lire** avant de commencer.

04 GRACE

~45 s

Slide 4 — Le besoin

Avant de rentrer dans la technique, je vais vous expliquer **le besoin**.

L'idée c'est qu'un patient veut une info médicale fiable et rapide.

Le problème, c'est qu'un LLM tout seul peut **halluciner** — il invente parfois des trucs faux.

Donc on voulait un chatbot qui réponde **uniquement à partir de sources officielles HAS et INCa**.

Et important : c'est **informatif**, on fait pas de diagnostic. C'est un choix qu'on assume.

05 GRACE

~45 s

Slide 5 — Pourquoi un LLM « tout fait »

Petite parenthèse pour répondre à une question qui revient souvent.

L'enjeu n'est pas de réinventer un LLM, mais **de savoir l'orchestrer**.

L'analogie c'est qu'on réinvente pas le moteur d'une voiture, on construit le châssis autour.

Le vrai boulot c'est : le pipeline RAG, l'orchestration backend, le prompt engineering, la comparaison de modèles. **C'est ça la SAE.**

06

GRACE

~30 s

Slide 6 — Stack technique

Voilà le récap de notre stack.

Frontend en **Streamlit**, backend **FastAPI** avec SQLAlchemy, auth JWT.

Pour le RAG on a **LangChain** (imposé par le sujet), et **ChromaDB** comme base vectorielle.

Embeddings avec all-MiniLM-L6-v2 — comparé à 2 autres.

Côté LLM : **Gemini 2.5 Flash** en prod, Mistral via OpenRouter en dev, avec un fallback.

On a aussi DuckDuckGo en bonus, et SQLite pour les sessions.

07

GRACE

~45 s

Slide 7 — Architecture générale

Et voilà l'architecture générale.

Le patient écrit dans Streamlit, ça part vers FastAPI qui gère l'auth et les sessions.

Le pipeline RAG va chercher dans ChromaDB qui a indexé tous nos PDFs HAS et INCa.

Si le score est mauvais, on bascule sur **DuckDuckGo** en filet de sécurité — premier bonus.

Le prompt assemblé part au LLM qui renvoie la réponse.

Et après, **Hermes** peut prendre le relais pour orienter vers un spécialiste — deuxième bonus.

💡 À noter : l'API n'est jamais exposée publiquement, tout passe par le réseau interne Docker.

PASSE LA PAROLE À : YOUSRA

■ Partie 2 — Traitement des PDFs & Pipeline RAG

Yousra · slides 8 à 13 · environ 3 minutes

08 YOUSRA

~10 s

Slide 8 — Transition « Partie 2 — Traitement des PDFs & Pipeline RAG »

Slide de transition.

09 YOUSRA

~50 s

Slide 9 — Étape 1 : indexation des documents

Donc moi je prends la suite pour vous expliquer comment on traite les PDFs.

On a 5 PDFs officiels qui couvrent 3 maladies : **diabète, Alzheimer et cancer du poumon**, sourcés HAS et INCa.

PyPDFLoader lit page par page. Ensuite on découpe avec RecursiveCharacterTextSplitter en morceaux de **500 caractères** avec un chevauchement de 50.

Chaque morceau passe par **MiniLM** qui le transforme en vecteur de **384 nombres**.

Et on stocke le tout dans ChromaDB. Au total ça fait **971 chunks indexés**.

💡 Important : cette étape se fait une seule fois, au démarrage. Pas à chaque requête.

10 YOUSRA

~30 s

Slide 10 — Pourquoi un chevauchement de 50 caractères ?

Alors pourquoi 50 caractères de chevauchement ?

Sans chevauchement, une phrase clé peut être coupée en deux entre 2 chunks. Par exemple « repose sur la metfor » / « mine en première intention » — la phrase complète est perdue.

Avec 50 caractères de recouvrement, le chunk suivant recule un peu avant de repartir. La phrase « repose sur la metformine » se retrouve **intacte dans au moins un chunk**.

Petit détail mais ça change la qualité du retrieval.

11

YOUSRA

~35 s

Slide 11 — Le rôle de MiniLM

Et MiniLM dans tout ça, c'est quoi son rôle exactement ?

C'est un modèle d'embeddings qui transforme n'importe quel texte en un **vecteur de 384 nombres**. Ce vecteur capture le *sens* du texte, pas juste les mots.

500 c'est la taille du chunk en caractères. **384** c'est la taille du vecteur — fixe peu importe la longueur du texte.

Exemple : « fièvre » et « température élevée » donnent des vecteurs proches, parce que sémantiquement c'est la même idée.

12

YOUSRA

~35 s

Slide 12 — Étape 2 : répondre à une question (RAG en action)

Et maintenant l'étape 2 : comment ça marche quand un patient pose une vraie question.

La question est convertie en vecteur avec **le même MiniLM**.

ChromaDB cherche les **4 chunks les plus proches** sémantiquement. Si le score est mauvais, on bascule sur la recherche web (notre bonus).

Sinon on assemble un prompt avec le contexte des chunks + l'historique de la conversation.

Et le LLM lit ce prompt et rédige la réponse finale en français.

13

YOUSRA

~40 s

Slide 13 — La similarité : comment on compare deux vecteurs

Petit point technique sur la similarité. Comment on compare deux vecteurs ?

On utilise la **distance L2** (distance euclidienne). Plus le score est **BAS**, plus c'est pertinent. C'est l'inverse de l'intuition habituelle.

On a fixé `RELEVANCE_THRESHOLD = 0.9`. Au-dessus, on considère qu'aucun chunk n'est assez pertinent, et on bascule sur le web pour ne pas répondre à côté.

La distance L2 c'est juste la racine de la somme des carrés des différences sur les 384 dimensions. De la géométrie classique.

PASSE LA PAROLE À : IMANE

■ Partie 3 — Comparaison encodeurs & Bonus DuckDuckGo

Imane · slides 14 à 18 · environ 3 minutes

14 IMANE

~10 s

Slide 14 — Transition « Partie 3 »

Slide de transition.

15 IMANE

~30 s

Slide 15 — Pourquoi comparer plusieurs encodeurs ?

Donc moi je prends la suite : comparaison d'encodeurs et bonus DuckDuckGo.

Pourquoi on a comparé plusieurs encodeurs ? Parce que c'est **explicitement demandé dans la SAE**, et aussi parce que le choix de l'encodeur change vraiment la qualité de la recherche sémantique.

On a testé 3 modèles sur les mêmes 5 PDFs : **MiniLM** en 384 dimensions, **CamemBERT** en 768 spécialisé français, et **mpnet** en 768 multilingue.

16 IMANE

~50 s

Slide 16 — Résultats de la comparaison

Et voilà les résultats.

MiniLM est le plus rapide : 71 secondes pour indexer nos 971 chunks. Qualité bonne mais parfois distrait par du bruit (citations bibliographiques).

CamemBERT, entraîné spécifiquement sur du français, capte mieux le sens médical. Mais 4 fois plus lent à indexer.

Mpnet est le plus pertinent des 3 sur nos questions test, aussi long à indexer.

On a choisi **MiniLM en prod pour la rapidité**. Les deux autres documentés comme alternatives plus précises.

17

IMANE

~40 s

Slide 17 — Bonus DuckDuckGo

Et maintenant le bonus DuckDuckGo.

Quand le RAG ne trouve rien de pertinent (score supérieur à notre seuil), on enrichit la réponse avec une recherche web.

Code tout simple : on appelle DDGS, on récupère les top 3, on injecte titre + extrait + URL dans le prompt.

Le web c'est un **filet de sécurité**, jamais une source par défaut. Le LLM badge explicitement quand l'info vient du web — l'utilisateur sait toujours d'où vient la réponse.

18

IMANE

~35 s

Slide 18 — Bug rencontré et corrigé

Petit bug qu'on a rencontré et corrigé.

Symptôme : un patient tape « j'ai de la fièvre », DuckDuckGo retournait des résultats Reverso et Collins — de la traduction, pas d'info médicale.

Cause : la phrase brute du patient ressemble à une phrase de manuel de langue.

Fix : `reformulate_search_query()` qui passe d'abord la phrase dans le LLM pour la transformer en requête médicale propre. « j'ai de la fièvre » devient « fièvre symptômes causes traitement ».

💡 Petit détail mais ça montre qu'on a vraiment testé et débogué l'appli.

PASSE LA PAROLE À : YANNIS

■ Partie 4 — Hermes, Déploiement & Démonstration

Yannis · slides 19 à 26 · environ 4 minutes

19 YANNIS

~10 s

Slide 19 — Transition « Partie 4 »

Slide de transition.

20 YANNIS

~45 s

Slide 20 — Hermes : orchestrateur multi-agents

Donc moi je prends la fin. Hermes, déploiement, et la démonstration.

Hermes c'est notre deuxième bonus, un **orchestrateur multi-agents**. Après une réponse du chatbot, le patient peut demander à voir un spécialiste.

On a 3 sous-agents : `recommend_specialist` qui analyse la conversation, `list_available_doctors` qui liste les créneaux, et `book_appointment` qui réserve.

Modèle Hermes 405B via OpenRouter en dev, fallback automatique. En prod : Gemini.

Ce qui rend Hermes intéressant : on va au-delà du Q&R, on propose **une action concrète** après le diagnostic informatif.

21 YANNIS

~40 s

Slide 21 — Hermes++ côté praticien (Telegram)

Bonus du bonus : **Hermes côté praticien** aussi.

Quand un patient prend rendez-vous, le médecin reçoit automatiquement une notification dans son groupe Telegram. Et il peut interroger Hermes en retour : `/aide`, `/rdv_aujourd'hui`, `/rdv_demain`, ou en langage naturel via Gemini.

Pour la démonstration on a 2 groupes Telegram : **Dr Lefebvre neurologue** et **Dr Benyahia endocrinologue**. Mais l'infra est prête pour N praticiens.

Cloisonnement strict : chaque groupe Telegram correspond à un seul médecin via le mapping `Doctor.telegram_chat_id`.

22

YANNIS

~30 s

Slide 22 — Sécurité & limites assumées

Côté sécurité, on a fait attention.

Pas de diagnostic : c'est rappelé dans chaque prompt envoyé au LLM.

Traçabilité : chaque réponse a un badge « documents officiels » ou « web », l'utilisateur sait toujours d'où vient l'info.

Limites assumées : périmètre limité à 3 pathologies, quota LLM gratuit ~1000 req/jour. ChromaDB et SQLite sur volume Docker persistant — pas de perte au redéploiement.

23

YANNIS

~35 s

Slide 23 — Déploiement en production

Déploiement. L'URL : **sae.amanawebagency.com**.

Caddy en reverse proxy avec HTTPS automatique Let's Encrypt.

Derrière Caddy, **3 containers Docker** : nginx pour le site landing à la racine, Streamlit pour le chatbot sur /app/, FastAPI totalement interne au réseau Docker — jamais exposé.

ChromaDB et SQLite sur volume persistant.

💡 *C'est de la vraie infra de production, pas un tier gratuit jetable. Mutualisé avec les autres services Amana.*

24

YANNIS

~35 s

Slide 24 — Intégration site + chatbot

On a aussi pas voulu balancer Streamlit nu à l'utilisateur.

Donc on a fait un **site landing MediGuide** à la racine, et le chatbot est intégré via une page /consulter avec iframe.

L'utilisateur arrive sur un vrai site contextualisé : il voit la mission, les sources, le disclaimer, puis lance l'assistant.

On a harmonisé tout le visuel — même palette teal, même font Inter — entre le site et le chatbot pour que ça fasse cohérent.

Slide 25 — Démo en direct

Pour la démo, 3 scénarios :

- 1 **Question dans le périmètre**, par exemple sur le diabète. Réponse avec badge « documents officiels ».
- 2 **Question hors périmètre**, genre la grippe. Bascule sur le web, badge « recherche web », sources citées.
- 3 **Hermes** : on clique sur le bouton, il recommande un spécialiste, on prend rendez-vous avec un vrai formulaire détaillé.

⚠ Préparer les 3 questions à l'avance dans un onglet ouvert. Si la connexion flanche, on a un backup en captures dans le rapport.

Slide 26 — Polish UX

Récap de la cohérence UX qu'on a soignée.

Cohérence visuelle : même palette et font partout.

Badges de sources sur chaque réponse.

Section « **Mes rendez-vous** » permanente dans la sidebar.

Auto-rename des sessions pour s'y retrouver entre plusieurs conversations.

PASSE LA PAROLE À : TOUS

■ Conclusion & Questions

Tous les 4 ensemble · slides 27 et 28

27

TOUS

~30 s

Slide 27 — Bilan des compétences mobilisées

Bilan des compétences mobilisées.

Full stack : FastAPI + Streamlit + JWT + SQLAlchemy.

IA / LLM : RAG, embeddings, prompt engineering, comparaison de modèles.

Esprit critique : on a réfléchi aux limites du RAG, au seuil de pertinence, à la sécurité, au fait qu'on ne fait pas de diagnostic.

DevOps : Docker, reverse proxy Caddy, déploiement réel en production.

28

TOUS · Merci

~30 s

Slide 28 — Merci / Questions

Voilà, on en a fini. **Merci pour votre attention.**

On reste dispo pour toutes vos questions, chacun sur sa partie.

Si vous voulez tester l'appli vous-même, l'URL est là, **sae.amanawebagency.com**, ou scannez le QR code à droite.

Notre dépôt GitHub est aussi disponible.

 Les 4 reviennent au centre pour les Q&R. Chacun répond aux questions sur sa partie ; les autres complètent si besoin.